System Design Interview — An Insider's Guide · Volume 2 · Chapter 9

S3-like Object Storage

설계 범위

설계 흐름

요구와 규모 → PUT / GET → Data Store

Placement · replication · EC → metadata / LIST

Versioning · multipart → GC와 공간 회수

자료 · 검토 범위

Chapter 모델: 공개 노트와 저자 공개 설명 대조

S3 API: AWS 공식 문서로 동작 확인

추가 설계: publish 실패 · local index · reader 보호

Garage / MinIO는 본문 뒤 보충입니다.
도서 원문 전체를 직접 대조한 자료는 아닙니다.

Block · File · Object

구분	접근 단위	Client가 다루는 추상화	대표 설계 관심사
Block	Block address	Volume과 raw block	Random I/O, filesystem 구성
File	Path · file	Directory와 file operation	공유, 권한, namespace
Object	Bucket · key	Bytes와 metadata를 가진 object	Scale, durability, 저장 효율

Object Storage: REST API로 bucket과 key를 지정합니다. PUT은 새 bytes를 저장하고 해당 key가 가리키는 object를 바꿉니다.

Bucket · Key · Version · Object ID

Overwrite · Version

Bucket: **study**
Key: **photos/a.jpg**

PUT v1 Object ID A → payload A

PUT v2 Object ID B → payload B

식별자

(bucket, key) → 현재 object의 주소

Version ID → API에서 특정 이력 지정

Object ID → Data Store의 immutable bytes

Versioning-enabled bucket 예시.
v2 공개 후 current는 v2, v1 bytes는 보존합니다.
Version ID의 숫자·시간 순서로 current를 추론하지 않습니다.

요구사항

기능 · Object 크기

Bucket · PUT · GET · DELETE · LIST · Versioning

수십 KB의 작은 object → 파일 관리 비용

수 GB 이상의 큰 object → 재전송 비용

설계 목표

연간 100 PB · 저장 효율

Durability 6 nines · 데이터 보존

Availability 4 nines · 요청 성공

목표 수치이며 달성 결과가 아닙니다.

용량 추정

Object 분포 · 평균 크기

소형 20% · 중형 60% · 대형 20%

대표 크기: 0.5 · 32 · 200 MB

가중 평균 59.3 MB

발표 가정: 1년간 신규 logical payload 100 PB.
Decimal 단위. Version 누적·삭제·overwrite는 제외합니다.

Object · Metadata 규모

100 PB ÷ 59.3 MB
약 16.9억 objects

1 KB/object
약 1.69 TB metadata

Metadata 수치는 record 본체 추정입니다.
Index · replica · version · 운영 여유는 별도.
공개 노트의 40% 계산은 capacity.md에 분리했습니다.

Workload · QPS · IOPS

유입량 가정

연간 100 PB 전량 신규 PUT · 균등 유입

평균 약 **53.47 objects/s · 3.17 GB/s**

공개 노트의 QPS가 아닌 계산 예시입니다. GET 비율 · peak · multipart 요청은 추가 입력입니다.

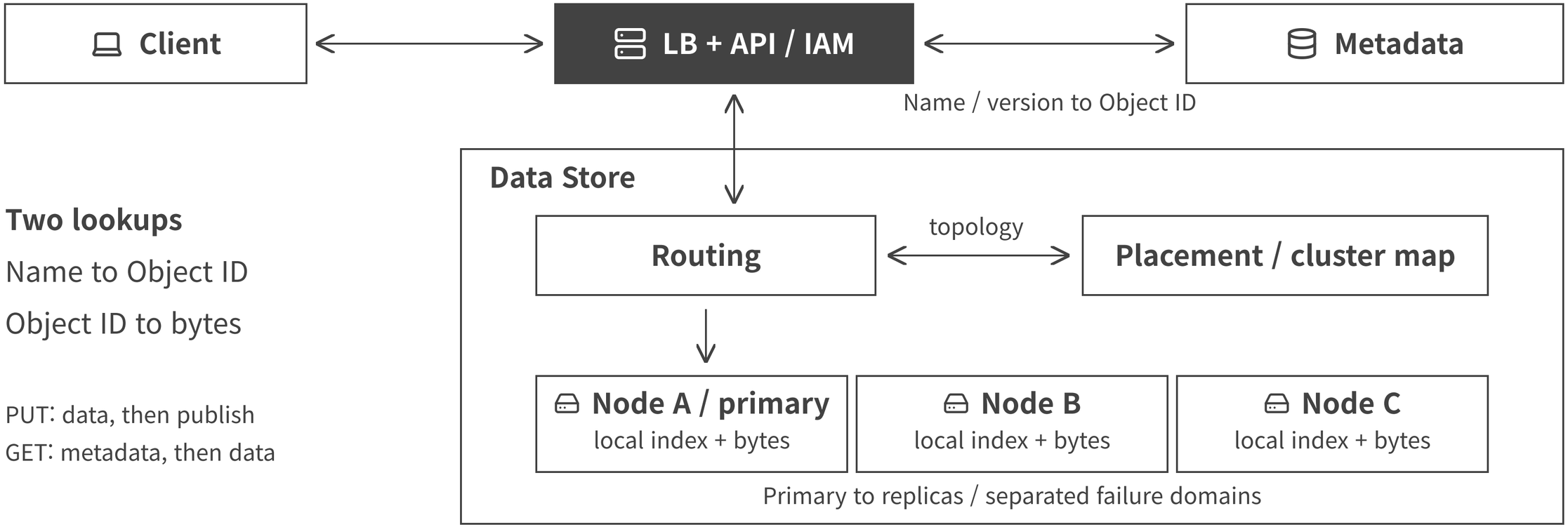
병목 · 확장

HDD random I/O **100–150 IOPS**는 노트의 가정입니다.

API / Routing → 요청 수 · 동시 연결
Data Node → 용량 · IOPS · bytes/s

Packing은 쓰기 seek를 줄이고, repair · compaction용 I/O 여유를 남깁니다.

High-level Architecture



Two lookups

Name to Object ID

Object ID to bytes

PUT: data, then publish

GET: metadata, then data

Metadata · Local Index

Namespace · Metadata service

(bucket, key)
→ current version
→ Object ID

Bucket: owner · policy · versioning mode

Version: size · checksum · created_at

Current와 LIST entry는 같은 shard / transaction에서 공개합니다.

Location · Data Node

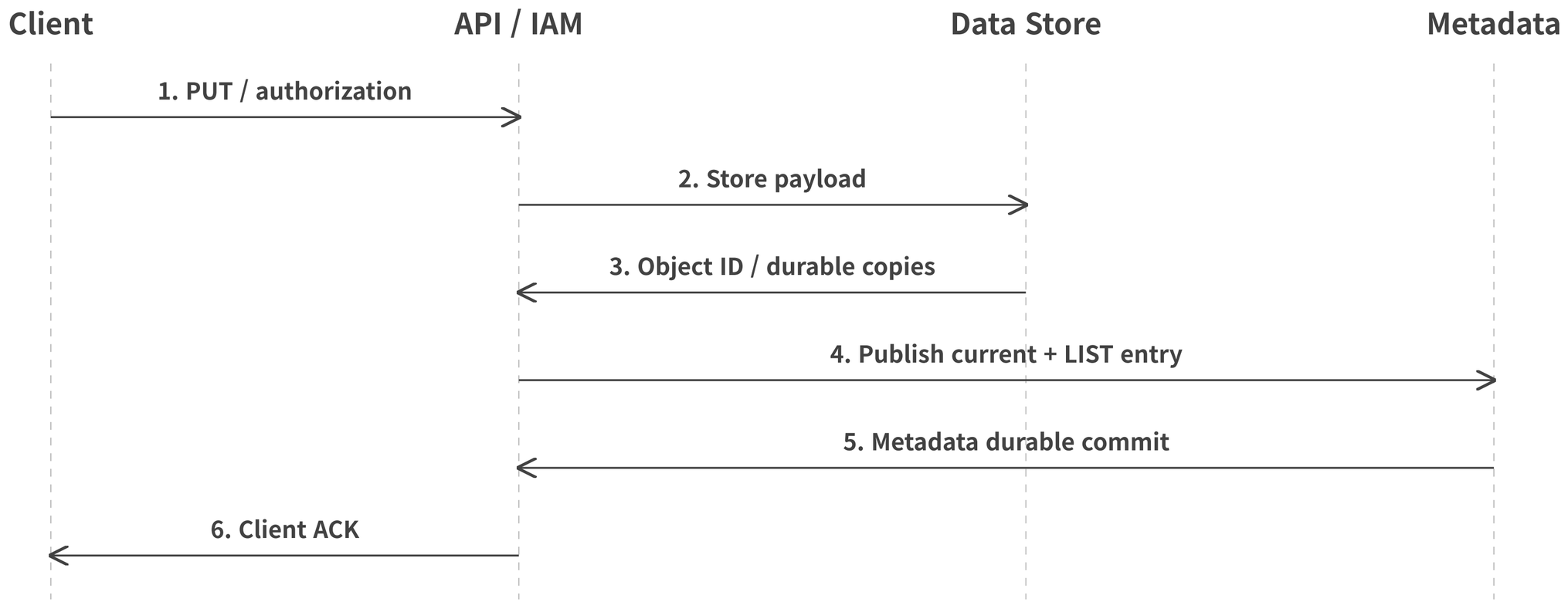
Object ID
→ segment ID
→ offset · length

Node마다 자신의 저장 위치를 관리합니다.

Compaction은 이 local map만 변경.

Current version과 API 주소는 바뀌지 않습니다.
두 map을 하나의 전역 DB transaction으로 묶지 않습니다.

Upload · Publish · ACK



Data Store response is internal. Client success follows the metadata commit.

Upload Failure

중단 시점 · 실패 상태

중단 시점	상태 · 처리
Payload 저장 중	Partial data · 아직 미공개 Retry · 불완전한 tail / part 정리
저장 완료 · publish 전	Orphan bytes · GET에 미노출 In-flight publish 확인 후 GC
Publish 결과 불명 또는 ACK 유실	Commit돼도 client는 모를 수 있음 완료 기록 조회 · retry 의미 정의

Payload 보호 + Metadata durable commit → Client ACK
Operation ID / 완료 기록은 설계 후보이며 S3 exactly-once 보장이 아닙니다.

Read · Modify · Write

1 A · B가 같은 ETag E0 읽음

2 A: 수정본 PUT → E1

3 B: E0를 바탕으로 수정본 PUT

무조건 PUT
A의 수정을 덮을 수 있음

If-Match: E0
현재 E1과 불일치 → 412

Atomic publish ≠ lost-update 방지

Download

1. Namespace

study / photos/a.jpg
→ current v2
→ Object ID B

→

2. Placement

Object ID B
→ replica group
→ 읽을 node

→

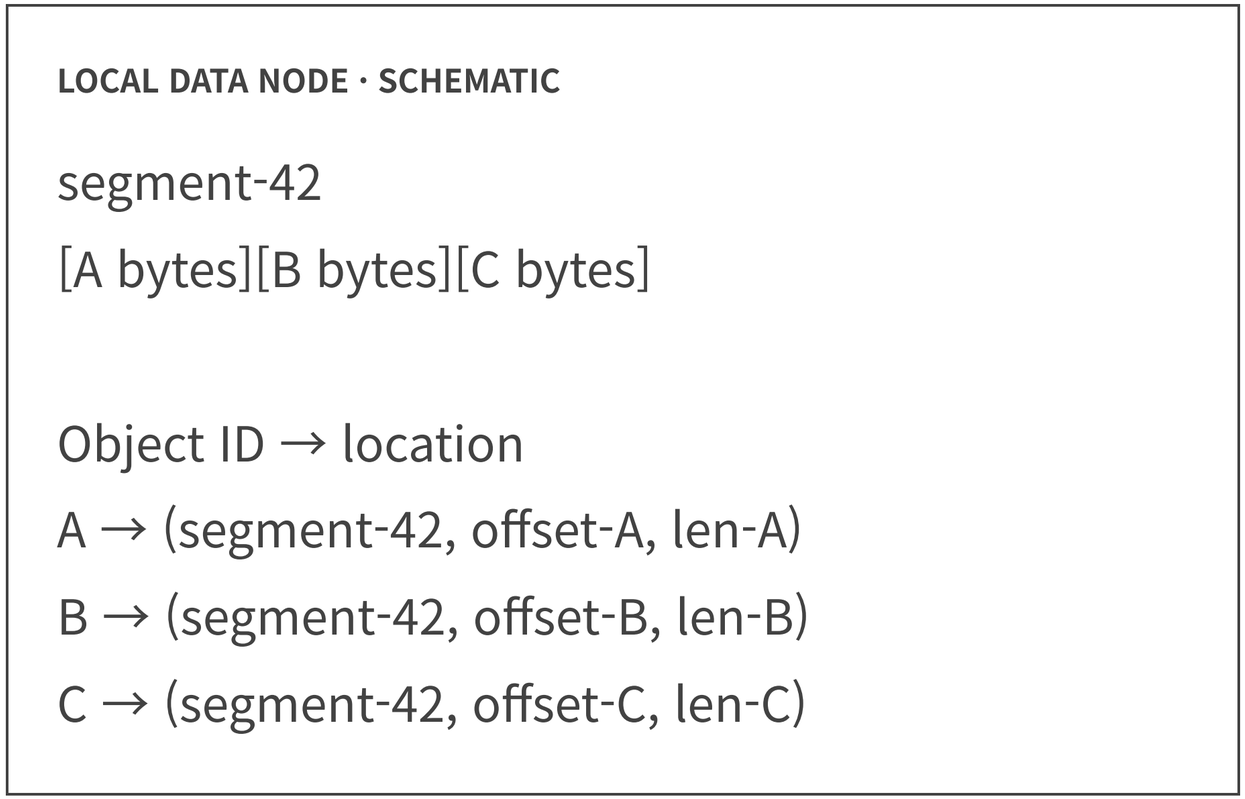
3. Local index

Object ID B
→ segment · offset
→ bytes streaming

먼저 읽을 version을 정하고, 그 version의 bytes를 찾습니다.

동시 overwrite가 payload를 제자리 수정하지 않도록 합니다. GC / compaction은 진행 중인 reader를 보호해야 합니다.

Small Object · Segment Packing



Append · Seal · Compact

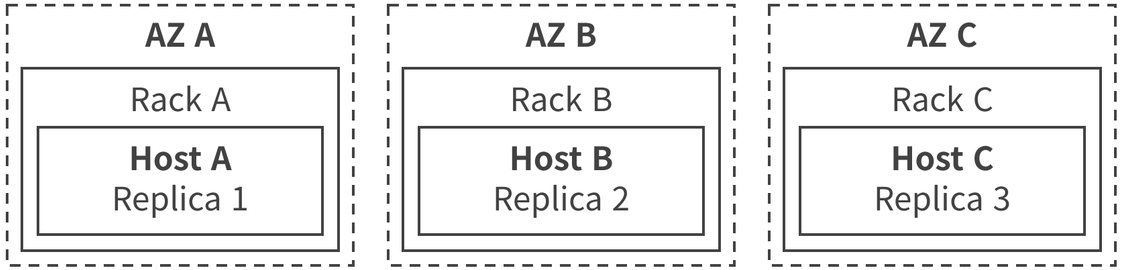
계산 가정: 개수의 20%가 1 MB 미만.
Segment로 block 낭비와 inode
부담을 줄입니다.

Append로 쓰기 seek 부담을 줄입니다.
읽기는 local index로 offset을 찾습니다.

Segment에 bytes를 안전하게 저장한 뒤
index를 공개합니다.

Placement · Failure Domain

Object Placement



Placement가 topology와 node 상태를 관리합니다.

Consistent hashing + virtual nodes
→ node 증감 시 이동 범위·부하 분산

Heartbeat 누락 → 장애 **의심**
실제 손실과 일시 partition은 다릅니다.

Placement Control

Host · rack · AZ에 사본 분산

Placement map은 Raft / Paxos 등
consensus로 일관되게 관리

추가 설계: map epoch 검사와 fencing
→ 오래된 위치를 가진 writer 차단

Hash 분산만으로 장애 격리나 stale writer 차단이 해결되지는 않습니다.

Replication · ACK

세 사본 중 대기 조건	요청 지연·가용성	공개 후 책임
3 durable copies	가장 느린 replica를 기다림	배치·손상 탐지·지속적 repair
2 durable copies · 설계 예시	한 replica가 늦어도 진행 가능	부족한 copy 보충 · 해당 version에서 읽기

ACK 개수 ≠ Consistency ≠ Disk flush

책 모델은 세 사본 완료를 기본으로 설명합니다.

일부 사본만 기다리는 설계는 metadata ordering · stale writer 차단 · 읽을 replica 선택까지 함께 정해야 합니다.

Erasure Coding

RS 8 + 4 · AZ 배치 비교

배치 A · 4 / 4 / 4



정상 shard 8개 → 복원 가능

배치 B · 6 / 3 / 3



정상 shard 6개 → 현재 복원 불가

같은 저장 배수 · 다른 내성

정상 shard **8개**가 복원 기준입니다.
같은 AZ의 shard는 함께 끊길 수 있습니다.

---

100 PB logical payload

EC 8 + 4 → **150** PB

Replica ×3 → **300** PB

Protection only · 1.5× vs 3×

Encoding CPU · multi-shard I/O
Degraded read · rebuild 비용

학습용 배치 · 남은 shard는 정상·접근 가능
복원 가능 ≠ 요청 가용성 / write quorum
일시적 단절 ≠ 영구 손실

Durability · Availability

Durability · 데이터 보존

6 nines 목표 → 장애 범위 분리 · repair Replica · EC로 손실에 대비합니다.

같은 장애에 여러 사본이 손실될 수 있습니다. Failure domain과 복구 시간을 함께 평가합니다.

Availability · 요청 성공

4 nines 목표 → 요청 경로의 중복화 데이터가 남아 있어도 metadata나 network 장애로 요청은 실패할 수 있습니다.

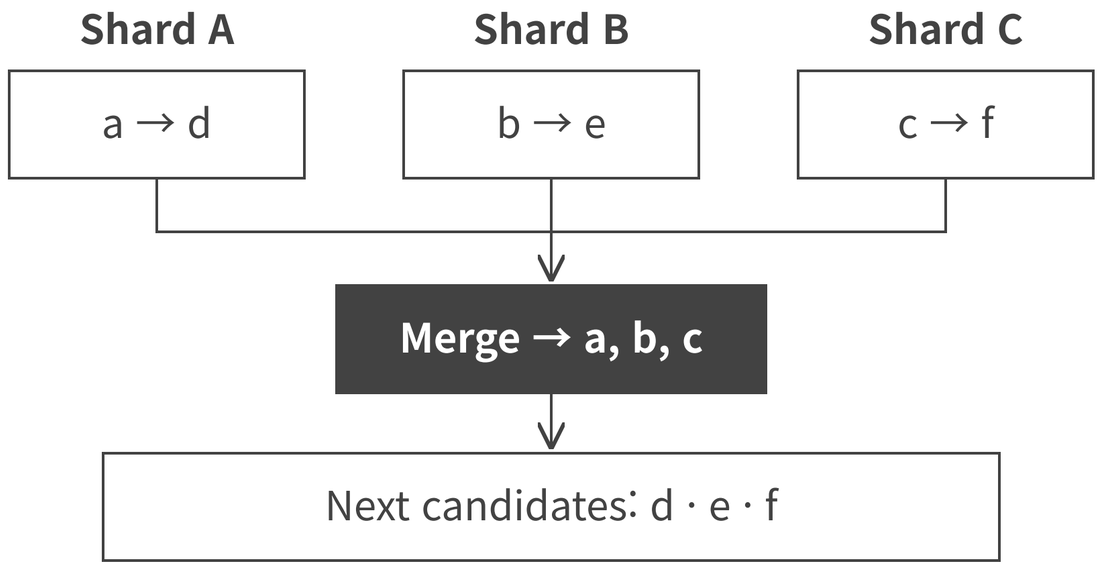
GET/PUT 성공률과 p99 latency를 측정합니다.

Durability 목표와 Availability 목표는
별도로 검증합니다.

LIST · Sharding

Prefix · Sorted Merge

LIST photos/ · page size 3



설계 예시: 현재 후보 a · b · c를 소비한 뒤
각 shard의 다음 후보 d · e · f에서 이어갑니다.

Publish · Consistency

Hash(bucket, key) 분산.
각 shard에 (bucket, key) 정렬 index.

한 key의 current + LIST entry
→ 같은 shard transaction으로 공개

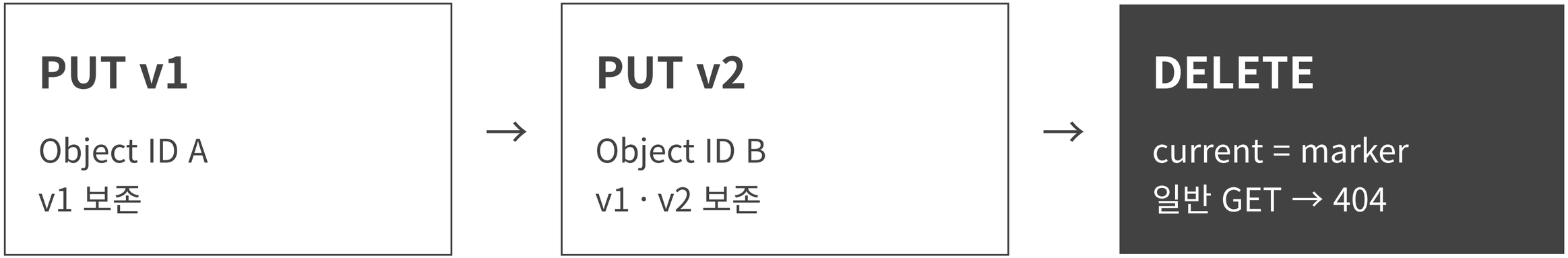
성공한 PUT 뒤 시작한 GET / LIST는 이후 변경이 없다면 새 상태를 반영합니다.

필요한 shard가 실패하면?

기본안: LIST 실패
부분 결과를 완전한 목록처럼 반환하지 않음

Cursor는 설계 예시 · AWS token 내부 구조 아님.
Token ≠ 여러 page의 고정 snapshot
Range: prefix locality ↔ hot prefix split

Versioning · Delete Marker



현재 목록에서 사라져도, 이전 data version은 live입니다.

Version ID로 이전 bytes 조회 · 영구 삭제 / 보존 종료는 별도

Versioning-enabled bucket에서 Version ID 없는 DELETE 예시. Marker의 Version ID를 직접 지정한 GET은 405입니다.

Multipart Upload

Initiate

Upload ID와
checksum 방식 설정

Upload parts

병렬 전송 · 실패한
part만 retry

Complete

Part 목록 확인 후
완성된 object 공개

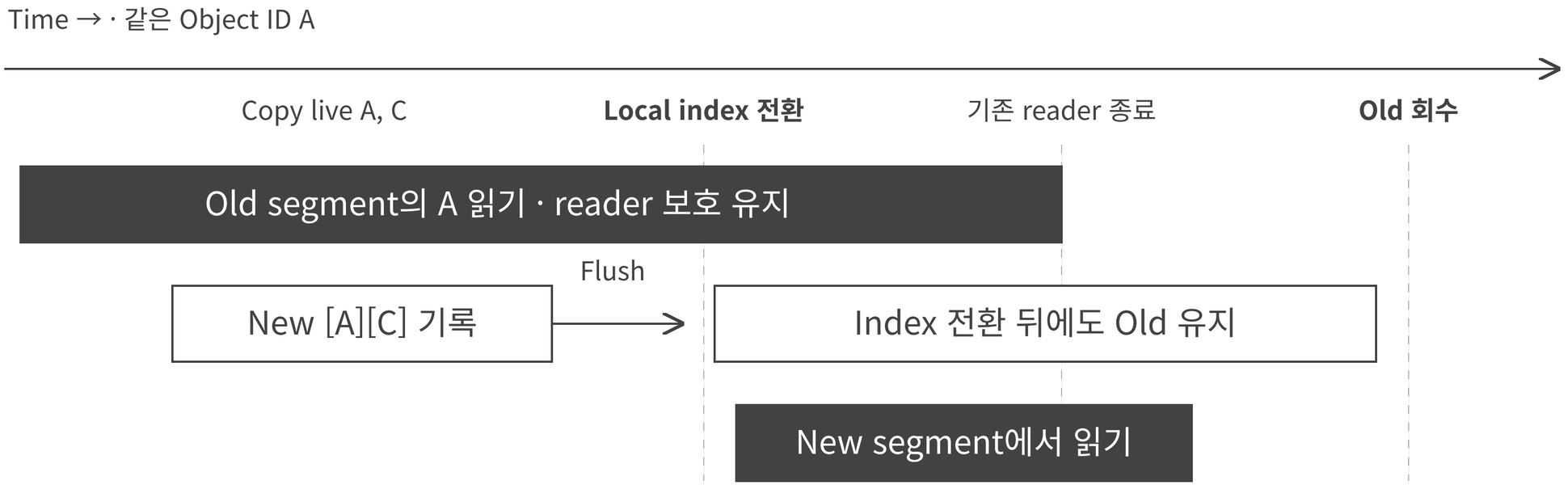
중단 경로: Abort

완료하지 않을 upload는 Abort로 part 공간을 정리합니다.
대형 object는 실패한 part만 retry하고, Complete에서 결합 · 검증합니다.

HTTP 200 header ≠ Complete 최종 성공

200 헤더 뒤 본문에 오류가 포함될 수 있습니다.
최종 본문 / SDK 결과를 확인합니다.

Garbage Collection · Compaction



Location만 변경

Old [A][dead B][C] → New [A][C]

Namespace current · Object ID는 유지

GC Safety

Retained version은 live · in-flight publish 확인

회수 eligibility와 node-local 이동 절차를 구분

추가 설계 예시: reader 등록 / 위치 해석과 회수 사이의 race도 차단해야 합니다. Index 전환만으로 old segment를 즉시 지우지 않습니다.

Repair · Rebalance

Copy → **Verify** → **Placement 전환** → **Old copy 회수**

목적	보호할 대상	확인할 비용
Repair · 부족한 사본 복원	Checksum · 남은 정상 copy	Repair backlog · 복구 시간
Rebalance · node 구성 변경	Old writer fencing · 진행 중인 reader	Network · IOPS · rebuild 중 p99

추가 설계 흐름: 새 위치가 준비되기 전 기존 copy를 버리지 않습니다. Epoch 검사와 reader 보호는 전환 경계에서 실행되어야 합니다.

Security · Integrity

접근 제어 · 암호화

Authentication · Authorization

누가 어떤 bucket / key에 접근할 수 있는지 검사합니다.

HTTPS · at-rest encryption · key와 접근 기록 관리

손상 탐지 · 복구

Upload / Download checksum 검증

정기 scrubbing으로 저장 bytes의 corruption 탐지

손상은 replica / EC shard에서 복구합니다.

암호화와 checksum은 서로 다른 책임입니다.

설계 요약

저장 효율 ↔ 복구 비용	Replication / EC · failure domain · repair 여유
조회 분산 ↔ 목록 비용	Hash sharding / prefix LIST · fan-out / range split
빠른 논리 삭제 ↔ 안전한 공간 회수	Version 보존 · local index 전환 · in-flight reader

Payload 조건 + Metadata commit → ACK · 보존 조건 해제 → GC

다음 4장은 구현 비교 보충입니다. 본문 토론만 할 때는 마지막 ‘생각할 거리’로 이동할 수 있습니다.

보충 · 저장 구조

논리적 책임	Garage	MinIO public snapshot
Namespace / metadata	metadata_dir · embedded DB	Object 경로 · xl.meta journal
Payload 보호	Hash-addressed block replicas	Object erasure set · shards
작은 object	Metadata Inline bytes	xl.meta Inline shard data
S3 Versioning	공식 compatibility 문서상 미지원	Pinned source의 versioned PUT

Logical separation ≠ 별도 DB service 필수 · Inline ≠ Segment packing

내부 version state가 있다는 것과 S3의 version 조회·복구 API를 지원한다는 것은 다릅니다.

보충 · Garage

Replication · Placement

Object → blocks → block hash 기반 배치 · dedup

Replica를 가능한 서로 다른 zone에 분산

큰 PUT: Uploading → blocks 전송 → Complete

Capacity · zone 기반 배치 · node 변경 후 rebalance

Quorum · 영속화

Factor 3 · consistent: **W=2 / R=2**

공식 문서의 read-after-write 보장

Degraded: W=2 / R=1 · 보장 약화

ACK 수와 `data_fsync` · `metadata_fsync`는 별도로 검토합니다.

보충 · MinIO

Erasure Set · 파일 구조

Object name hash → pool 안의 erasure set

Reed-Solomon shards를 set의 drives에 저장

`xl.meta` → versions · delete markers · part 정보

Quorum과 healing은 object의 set 단위입니다.

PUT 완료 경계

Temporary object encode → final `renameData`

Data K · parity M의 일반 object 경로:

Read K · Write K

K = M이면 **Write K+1** · operation별 조건은 별도입니다.

보충 · 구현 선택

요구 조건	먼저 검토할 구조	함께 확인할 비용
다른 node capacity · 여러 zone	Garage의 block replicas	Replica 공간 · API 호환 범위
EC 효율 · drive topology 계획	MinIO의 erasure sets	Degraded read · healing I/O
Version 복구 · 정책 이식	필요한 S3 API와 권한 모델	Retention 공간 · SDK integration

선택은 **요구 충족 → 배치와 복구 비용 → 실험** 순서입니다.

PUT 직후 GET/LIST · 동시 overwrite · 전원 장애 · rebuild 중 p99를 검증합니다.

참고문헌 · 모델 · API

Alex Xu - S3-like storage system - https://blog.bytebytego.com/p/design-a-s3-like-storage-system
ByteByteGo · Volume 2 목차 — https://blog.bytebytego.com/p/system-design-interview-books-volume
Public Book Notes · Object Storage · 공개 2차 자료 — https://books.dwf.dev/docs/system-design/c25
AWS · S3와 consistency — https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html
AWS · Multipart — https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html
AWS · Object integrity — https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity.html
AWS · Delete markers — https://docs.aws.amazon.com/AmazonS3/latest/userguide/DeleteMarker.html
Ceph · EC — https://docs.ceph.com/en/squid/rados/operations/erasure-code/
AWS · Conditional writes — https://docs.aws.amazon.com/AmazonS3/latest/userguide/conditional-writes.html
AWS · CompleteMultipartUpload — https://docs.aws.amazon.com/AmazonS3/latest/API/API_CompleteMultipartUpload.html

참고문헌 · 오픈소스

Garage · Configuration — https://garagehq.deuxfleurs.fr/documentation/reference-manual/configuration
Garage · Features — https://garagehq.deuxfleurs.fr/documentation/reference-manual/features
Garage · S3 Compatibility — https://garagehq.deuxfleurs.fr/documentation/reference-manual/s3-compatibility
Garage · pinned source — https://git.deuxfleurs.fr/Deuxfleurs/garage/src/commit/ced85f86a0feb81cb86e62d8bc72ae028ed41bc5
MinIO public server · pinned source — https://github.com/minio/minio/tree/7aac2a2c5b7c882e68c1ce017d8256be2feea27f
분석 경로: Garage `object_table.rs` · `put.rs` / MinIO `DESIGN.md` · `xl-storage-format-v2.go` · `erasure-object.go` · `erasure-metadata.go`

토론

- Zone 장애 중에도 PUT을 받아야 한다면, consistency와 손실 위험을 어디까지 허용할까요?
- EC가 rebuild 중 p99를 악화한다면, parity와 복구 여유를 어떻게 정할까요?
- Version 복구와 bucket policy가 필수라면, 어떤 호환 조건부터 확인해야 할까요?
- PUT 성공 직후 동시 전원 장애가 발생하면, 무엇을 영속화의 증거로 삼을 수 있을까요?